Group Presentation

Taking Ada as a case study

Group 6: Felix 202316801101
Even 202316801105
Alaska 202316801109

CSCU9Y4 Programming Languages Principles

1. Ada’s background
2. Language property and learning experiences
3. Application and historical impact

Ada’s background

Part 1

Basic Definition

- Positioning:

  A language designed for large systems.

- Property:

  Clear structure & Strong typing for early error detection

- Advantages:

  Easy review; Early error detection; Long-term maintenance.

- Mechanism:

  Visible Declarations; Block Structure (Barnes, 2007).

Ada's history

Ada's Background (1970s)

1. Systems were getting larger, more complex.
2. Become important control components in many areas
3. Small errors → safety problems

(Carlson et al., 1980; Ichbiah et al., 1979).

Policy Promotion

1. Large projects used different languages, dialects, local coding habits.

2.US Department of Defense set up revolutions. Goal: Reduce the cost and confusion in defense system caused by that mix (Whitaker, 1993)

Educational value of Ada

Core Challenges Remain

Still need to determine:

1. **What the language is intended to convey.**
2. **What mistakes need to be avoided.**

(Ray et al., 2017)

Modern Languages Disadvantages

1. **Low readability.**
2. **Expensive error cost.**

(Berger et al., 2019)

Features & Experience

Part 2

Key Features

Refuse ambiguity;

Strongly typed;

Narrow types with ranges or subtypes. (Barnes, 2007).

```
for I in 1 .. 10 loop
   Sum := Sum + I;
end loop;
```

Visible Structure;

Keeps state change and ranges explicit.

Tools, Compile-run Process & One Very Small Code Example

```
with Ada.Text_IO; use Ada.Text_IO;

procedure Sum_Loop is
   Sum : Integer := 0;
begin
   for I in 1 .. 10 loop
      Sum := Sum + I;
   end loop;

   Put_Line("Sum =" & Integer'Image(Sum));
end Sum_Loop;
```

An enclosing procedure;

Following a declaration;

Define the loop scope;

Update Sum = Sum + I;

End visibly;

Toolchain: GNAT

Learning Experience Conclusion

- **First Reaction:**

  Slow & Formal;

  Kept asking for surrounding structure and modules.

- **Attitude Shift:**

  Fussy parts are key to readability and long-maintenance.

  Help to find errors early & Make the code easier to maintain.

  (Gao, Bird and Barr, 2017; Bogner and Merkel, 2022)

Brief Comparison with Python

(Fast initial development (Sward, 2010)) ***Python***

Go straight to the task;
Creating a variable & a loop range;
Sum the values;
Print result.

```python
total = 0

for i in range(1, 11):
    total += i

print("Sum =", total)
```

Ada ***(Saving cost for checking & fixing bugs (Sward, 2010))***

Reduces workload in different ways
Makes all steps more visible.
(Prechelt, 2000; Ray et al., 2017)

```ada
with Ada.Text_IO; use Ada.Text_IO;

procedure Sum_Loop is
   Sum : Integer := 0;
begin
   for I in 1 .. 10 loop
      Sum := Sum + I;
   end loop;

   Put_Line("Sum =" & Integer'Image(Sum));
end Sum_Loop;
```

Application and Historical Impact

Part 3

```
package Ada is
    type Task_Id is private;
    procedure Start (T : Task_Id);
    task type Worker is
        entry Execute;
    end Worker;
end Ada;
```

Application Domains

- Avionics: navigation and real-time monitoring.
- Railway control and signaling: routing, interlocking, safe-mode control
- Space systems: command and sequences, instruments, fault management.
- Shared features: high complexity, long service life, low tolerance for hidden faults.

(Snavely et al., 2018; CENELEC, 2023; NASA, 2022)

Why Ada Fits These 3 Domains

Why Ada Fits Safety-Critical Systems

- Strong typing reduces accidental data mixing.
- Range constraints define acceptable values clearly.
- Visible block structure supports review and maintenance.
- Compile-time checking helps detect errors earlier.

(Barnes, 2007; Hoang et al., 2015)

Historical Influence and Wider Significance

- Ada is important for more than popularity.
- It treats correctness, clarity, and reviewability as language-design goals.
- It supports stronger interfaces between program units.
- It shows a key trade-off: security vs performance

(Whitaker, 1993; Sward, 2010)

Conclusion

- Ada came from a real engineering problem.
- Its strictness supports reliable, long-lived software.
- Emphasizes software readability, early error detection, and long-term maintenance.
- Its value is showing how language design can reduce risk.

Reference list

Barnes, J. (2007) Rationale for Ada 2005. Available at: https://www.adaic.org/resources/add_content/standards/05rat/Rationale05.pdf (Accessed: 18 April 2026).

Berger, E.D., Hollenbeck, C., Maj, P., Vitek, O. and Vitek, J. (2019). On the Impact of Programming Languages on Code Quality. ACM Transactions on Programming Languages and Systems, 41(4), pp.1–24. doi:https://doi.org/10.1145/3340571.

Bogner, J. and Merkel, M. (2022). To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub. [online] IEEE Xplore. doi:https://doi.org/10.1145/3524842.3528454.

Brandon, C. and Chapin, P. (2017). The Use of SPARK in a Complex Spacecraft. ACM SIGAda Ada Letters, 36(2), pp.18–21. doi:https://doi.org/10.1145/3092893.3092896.

Carlson, W.D., Druffel, L.E., Fisher, D.E. and Whitaker, W. (1980). Introducing Ada. 1980 Proceeding. doi:https://doi.org/10.1145/800176.809976.

CENELEC (2023) EN 50716:2023 Railway applications — Requirements for software development. Brussels: European Committee for Electrotechnical Standardization. Available at: https://standardsdevelopment.bsigroup.com/projects/2022-00241 (Accessed: 29 April 2026).

Gao, Z., Bird, C. and Barr, E.T. (2017). To Type or Not to Type: Quantifying Detectable Bugs in JavaScript. 2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE). doi:https://doi.org/10.1109/icse.2017.75.

Hoang, D., Moy, Y., Wallenburg, A. and Chapman, R. (2014). SPARK 2014 and GNATprove. International Journal on Software Tools for Technology Transfer, 17(6), pp.695–707. doi:https://doi.org/10.1007/s10009-014-0322-5.

Ichbiah, J., Bernd Krieg-Brueckner, Wichmann, B., Barnes, J., Olivier Roubine and Jean-Claude Heliard (1979). Rationale for the design of the Ada programming language. Sigplan Notices, 14(6b), pp.1–261. doi:https://doi.org/10.1145/956653.956654.

NASA (2022) NASA-STD-8739.8B: Software Assurance and Software Safety Standard. Washington, DC: National Aeronautics and Space Administration. Available at: https://standards.nasa.gov/sites/default/files/standards/NASA/B/0/NASA-STD-87398-Revision-B.pdf (Accessed: 29 April 2026).

Prechelt, L. (2000) ‘An empirical comparison of seven programming languages’, *Computer*, 33(10), pp. 23–29. doi:10.1109/2.876288.

Ray, B., Posnett, D., Devanbu, P. and Filkov, V. (2017). A large-scale study of programming languages and code quality in GitHub. *Communications of the ACM*, [online] 60(10), pp.91–100. doi:https://doi.org/10.1145/3126905.

Snavely, W., Meyers, C., Runyon, B., Inacio, C., Riley, M. and Lareau, J.D. (2018) ‘Flight software programming language selection: a security perspective’, 2018 AIAA SPACE and Astronautics Forum and Exposition. doi:10.2514/6.2018-5397.

Gao, Z., Bird, C. and Barr, E.T. (2017). To Type or Not to Type: Quantifying Detectable Bugs in JavaScript. *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. doi:https://doi.org/10.1109/icse.2017.75.

Sward, R.E. (2010). The rise, fall and persistence of Ada. *ACM SIGAda Ada Letters*, 30(3), p.71. doi:https://doi.org/10.1145/1879097.1879081.

Whitaker, W.A. (1993). Ada—the project. *ACM SIGPLAN Notices*, 28(3), pp.299–331. doi:https://doi.org/10.1145/155360.155376.

Thanks